MATPLOTLIB

Introduction

Matplotlib is a foundational graph plotting library in Python, widely used for data visualization. It was developed by John D. Hunter and is open-source, allowing free use by anyone.

Primarily written in Python, Matplotlib also includes some portions in C, Objective-C, and JavaScript to ensure compatibility across different platforms.

Installation

Installing Matplotlib is straightforward if Python and PIP are already set up on your system.

You can install it by running the following command:

```
C:\Users\Your Name>pip install matplotlib
```

If the installation command fails, you can use a Python distribution that comes with Matplotlib pre-installed, such as Anaconda or Spyder.

Importing Matplotlib

After installing Matplotlib, you can import it into your applications by adding the following statement:

```
import Matplotlib
```

Checking the Matplotlib Version

The version of Matplotlib can be checked using the __version__attribute.

Here’s an example:

```
import matplotlib

print(matplotlib.__version__)
```

This will display the installed version of Matplotlib.

Pyplot

Most of Matplotlib's functionality is found in the pyplot submodule, which is typically imported using the alias plt:

```
import matplotlib.pyplot as plt
```

Now, you can use the pyplot module by referring to it as plt.

Example

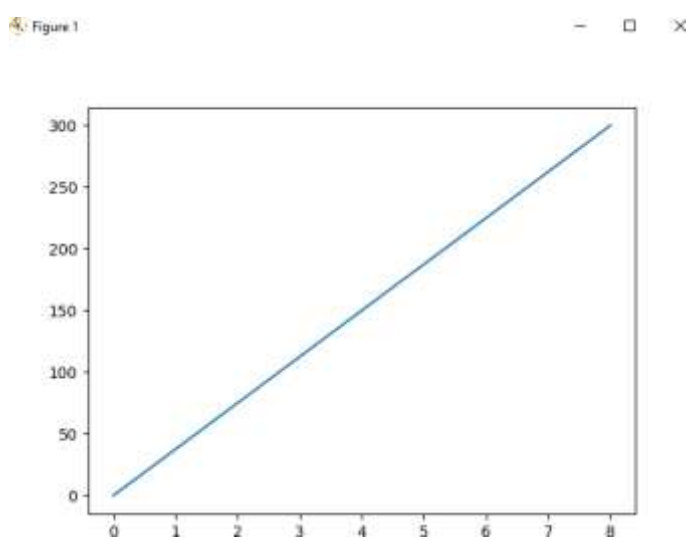
To draw a line in a diagram from position (0,0) to position (8,300), you can use the following code:

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 8])
ypoints = np.array([0, 300])

plt.plot(xpoints, ypoints)
plt.show()
```

This will plot a line between the specified points on the diagram.



Plotting

To plot points on a graph, the plot() function is used. This function typically connects the points with a line unless specified otherwise.

The plot() function requires two main parameters:

1. The first parameter is an array representing the x-axis values.
2. The second parameter is an array representing the corresponding y-axis values.

For example, to plot a line between the points (1, 7) and (2, 9), you would pass the arrays [1, 7] for the x-axis and [2, 9] for the y-axis to the plot() function.

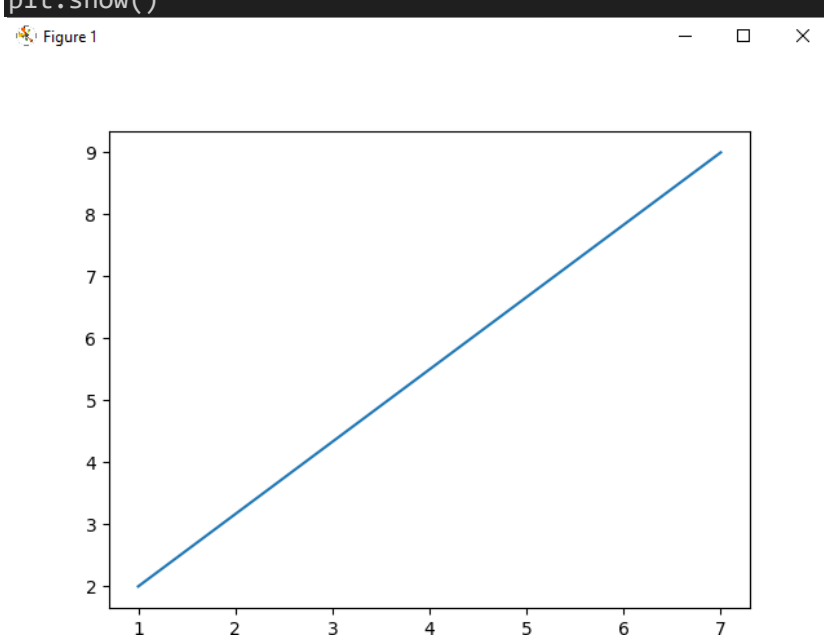
Example

Draw a line in a diagram from position (1, 3) to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 7])
ypoints = np.array([2, 9])

plt.plot(xpoints, ypoints)
plt.show()
```



The **x-axis** is the horizontal axis.

The **y-axis** is the vertical axis.

Plotting Without a Line

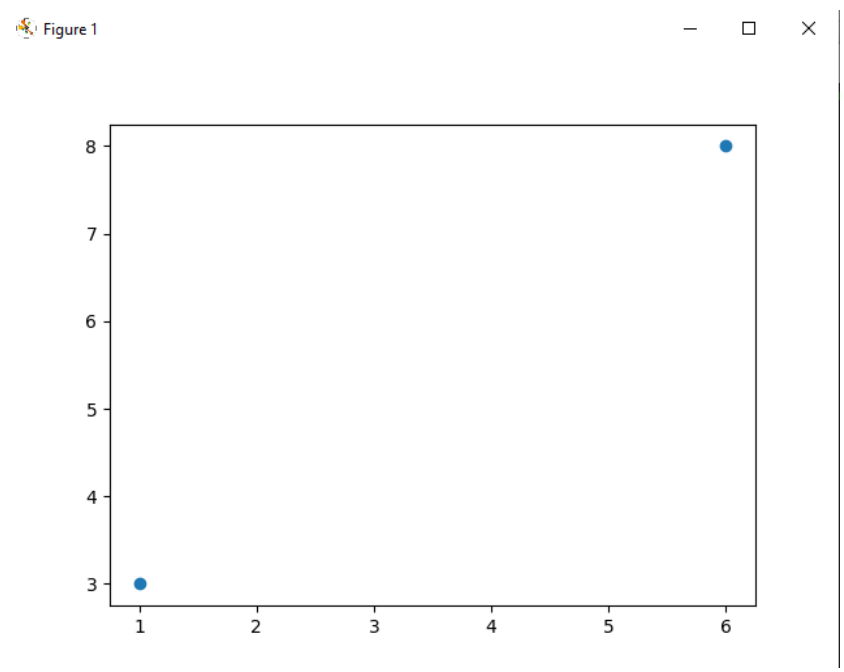
To display only the markers (without connecting lines), you can use the shortcut string notation 'o', which represents circular markers.

Example: To plot two points, one at (1, 6) and the other at (3, 8), use the following code:

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 6])
ypoints = np.array([3, 8])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```



Plotting Multiple Points

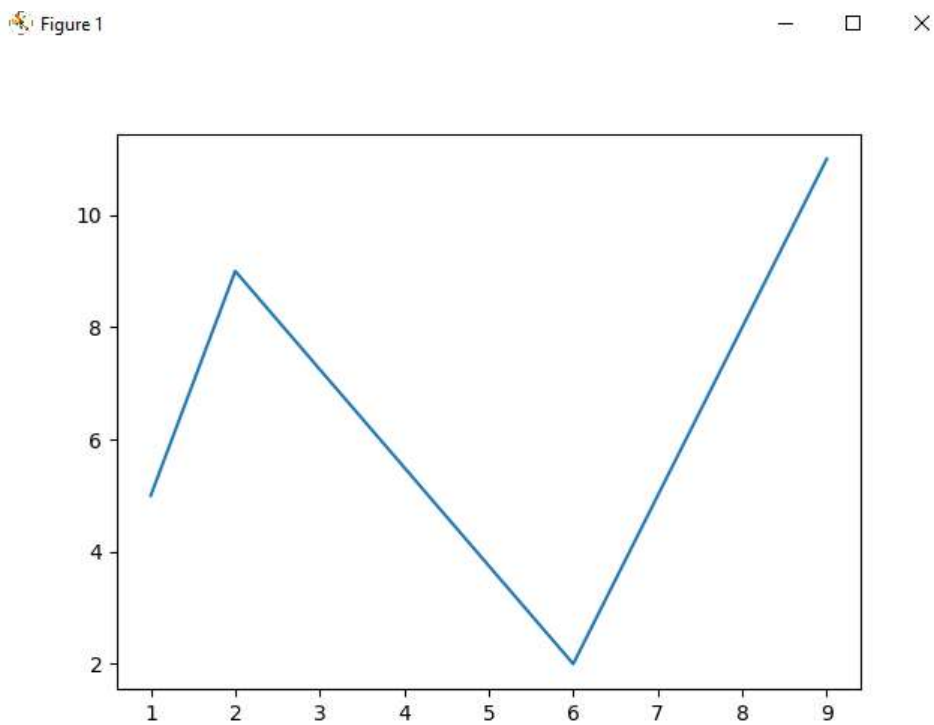
You can plot as many points as needed, just ensure the number of points on both the x-axis and y-axis are equal.

Example: To draw a line connecting the points (1, 5), (2, 9), (6, 2), and (9, 11), use the following code:

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 2, 6, 9])
ypoints = np.array([5, 9, 2, 11])

plt.plot(xpoints, ypoints)
plt.show()
```



Default X-Points

If the x-axis points are not provided, they will automatically take default values starting from 0, incrementing by 1 for each point, based on the number of y-axis points.

For example, using the same data as before but omitting the x-points:

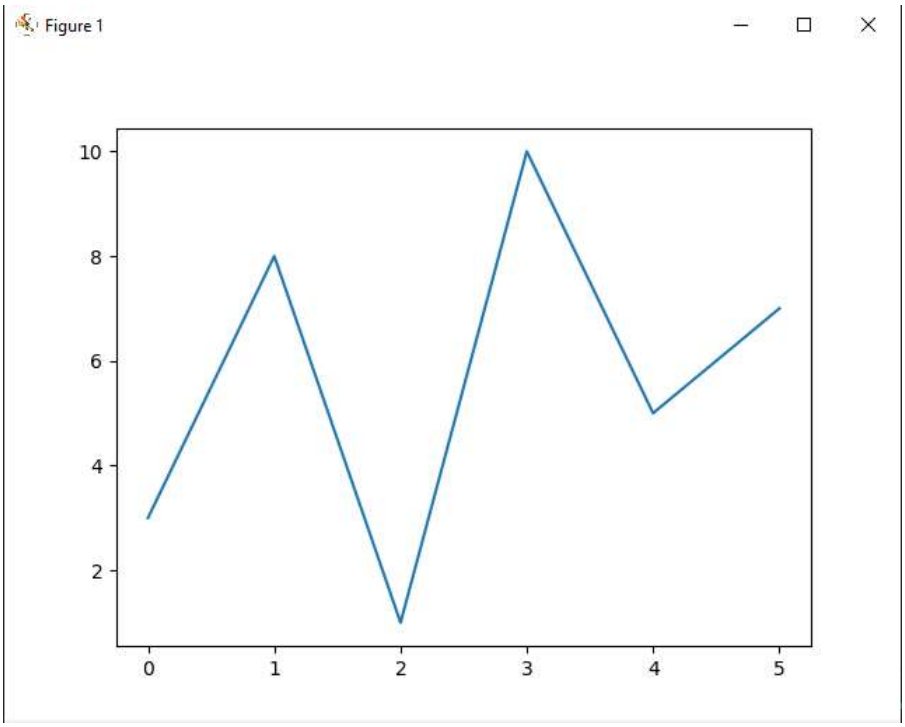
Example: Plotting without specifying x-points:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10, 5, 7])

plt.plot(ypoints)
plt.show()
```

The **x-points** in the example above are [0, 1, 2, 3, 4, 5].



Matplotlib Markers

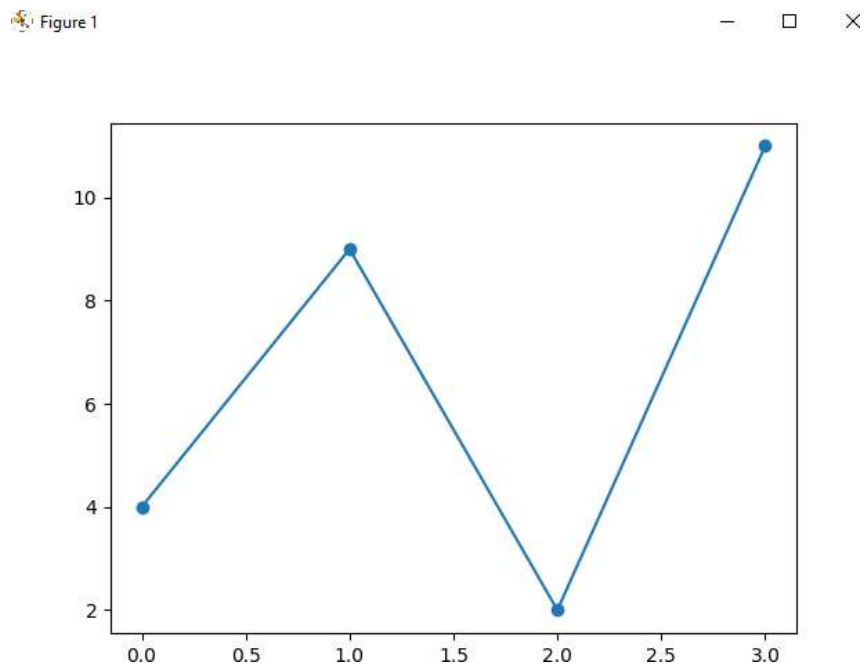
You can use the marker keyword argument to highlight each point with a specific marker symbol:

Mark each point with a circle:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([4, 9, 2, 11])
```

```
plt.plot(ypoints, marker = 'o')
plt.show()
```

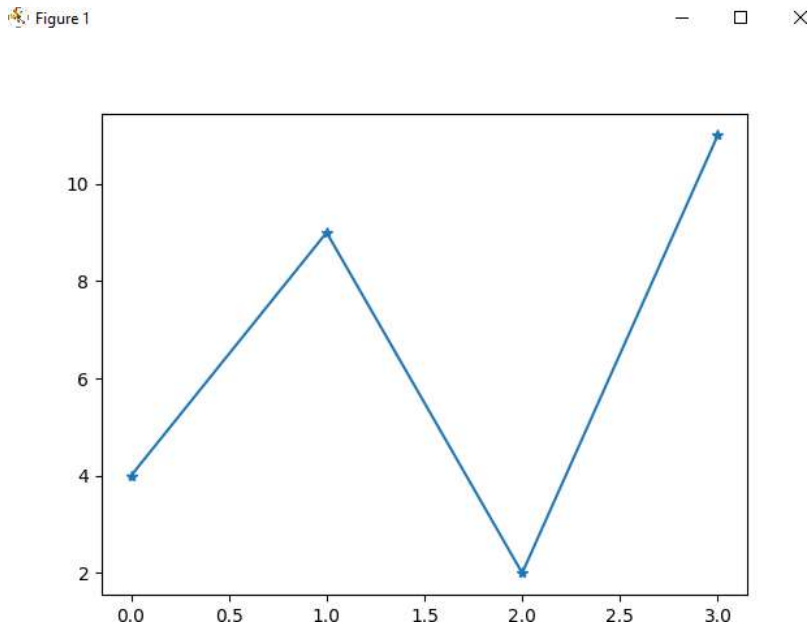


Mark each point with a *:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([4, 9, 2, 11])

plt.plot(ypoints, marker = '*')
plt.show()
```



Marker Reference

You can choose any of these markers:

Marker	**Description**
'o'	Circle
'*'	Star
'.'	Point
','	Pixel
'x'	X

'X'	X (filled)
'+'	Plus
'P'	Plus (filled)
's'	Square
'D'	Diamond
'd'	Diamond (thin)
'p'	Pentagon
'H'	Hexagon
'h'	Hexagon
'v'	Triangle Down
'^'	Triangle Up
'<'	Triangle Left
'1'	Tri Down

'2'	Tri Up
'3'	Tri Left
'4'	Tri Right
'|'	Vline
'_'	Hline

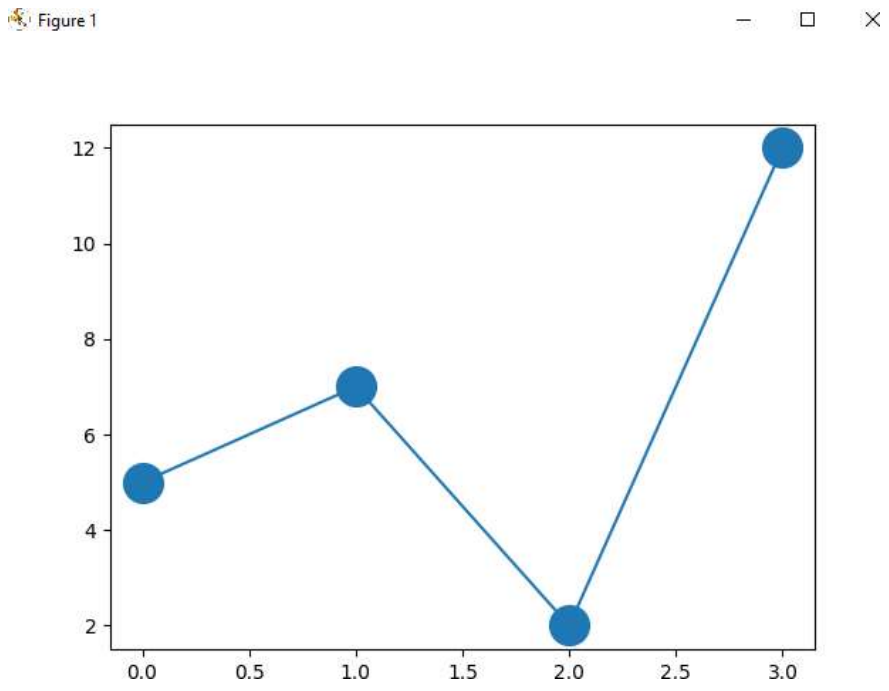
Marker Size

You can use the keyword argument markersize or the shorter version, ms to set the size of the markers:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([5, 7, 2, 12])

plt.plot(ypoints, marker = 'o', ms = 20)
plt.show()
```



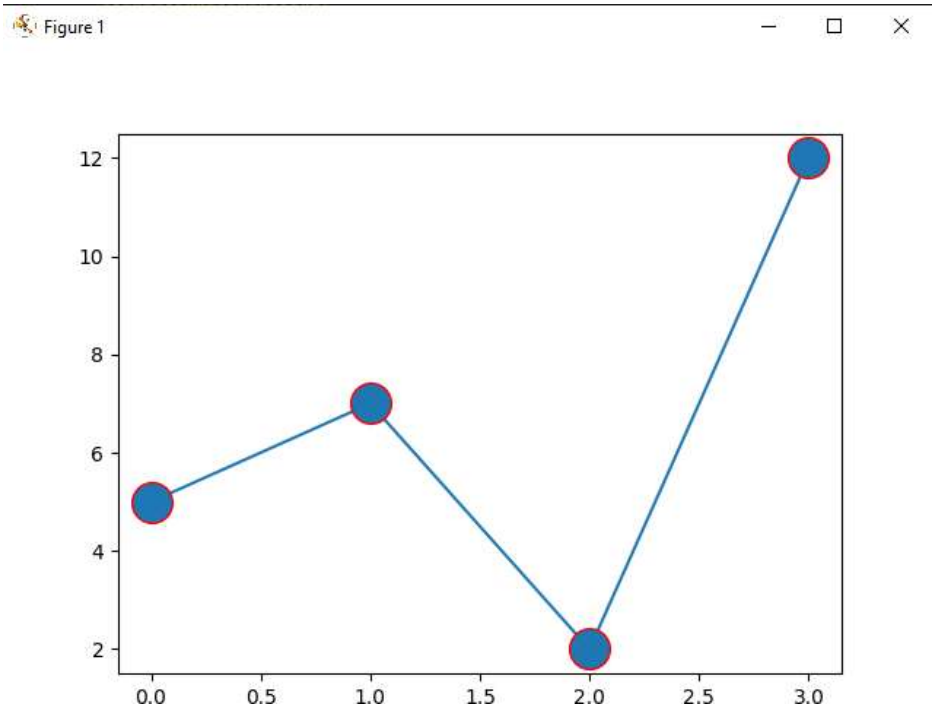
Marker Color

You can use the keyword argument markeredgecolor or the shorter mec to set the color of the *edge* of the markers:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([5, 7, 2, 12])

plt.plot(ypoints, marker = 'o', ms = 20, mec = 'r')
plt.show()
```



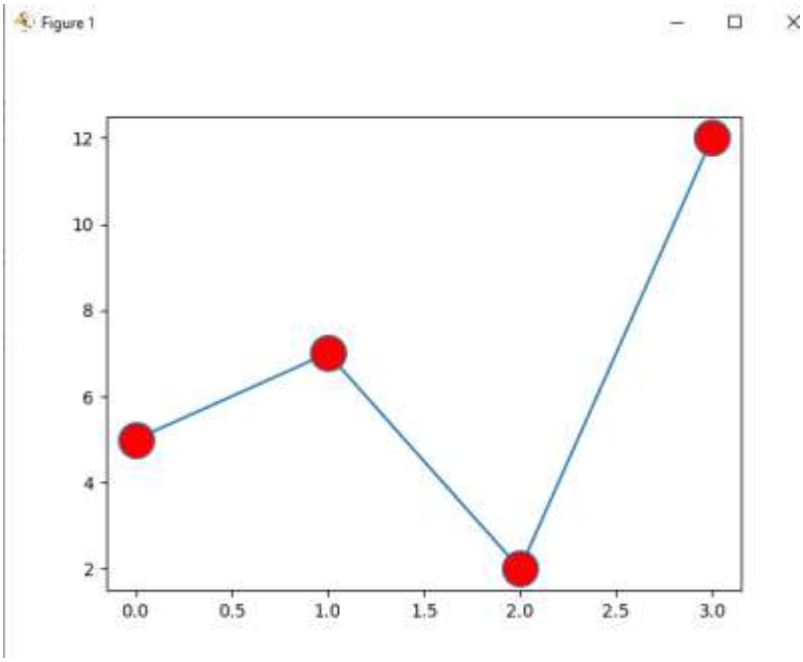
You can use the keyword argument `markerfacecolor` or the shorter `mfc` to set the color inside the edge of the markers:

Set the FACE color to red:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([5, 7, 2, 12])

plt.plot(ypoints, marker = 'o', ms = 20, mfc = 'r')
plt.show()
```



Use *both* the `mec` and `mfc` arguments to color the entire marker:

Set the color of both the *edge* and the *face* to red:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([5, 7, 2, 12])

plt.plot(ypoints, marker = 'o', ms = 20, mec = 'r', mfc = 'r')
plt.show()
```

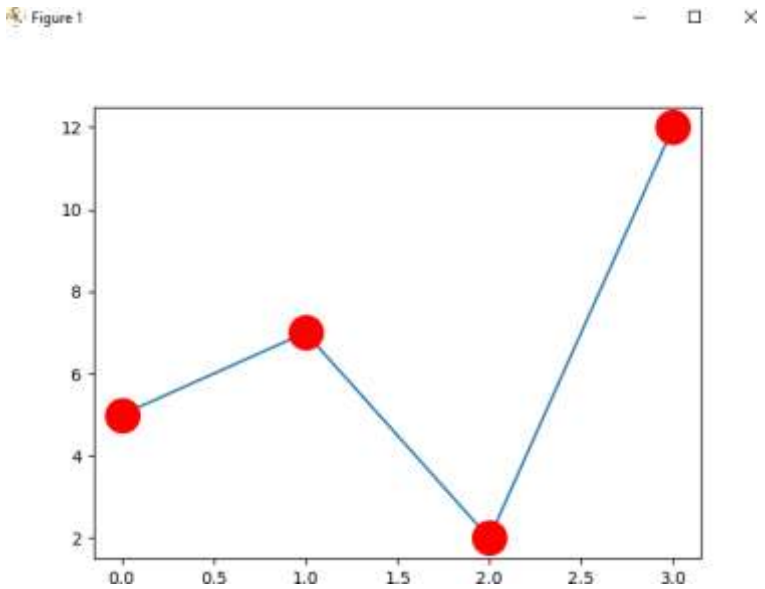


Figure 1
12
10
8
6
4
2
0.0
0.5
1.0
1.5
2.0
2.5
3.0